

Lezione — Modulo S2: Autenticazione

Corso: Lab Sicurezza Informatica T **Materiale:** Autenticazione_27_febbraio.pdf · approfondimento_PAM_-_il_framework_di_autenticazione_e_autorizzazione_.pdf **Pre-requisiti:** S1 (enumerazione — trovare le porte è il passo prima di attaccarle); SysAdmin 2A (utenti, /etc/shadow, permessi) **Nota esame:** questo modulo alimenta direttamente il **quiz teorico (40%)** — le distinzioni AAA/fattori/passivo-attivo/2FA-2SA sono domande tipiche. Penalità per risposta sbagliata: se non sei sicuro, lascia in bianco.

Come leggere questa lezione

Il filo conduttore è un'unica domanda: *come fa un sistema a sapere davvero chi sei?* I ganci concreti sono tre file/sistemi che già conosci: `/etc/shadow` (dove vive il segreto sul lato server), `/etc/pam.d/` (chi decide come il sistema verifica quel segreto), e `~/.ssh/authorized_keys` (dove vive l'autenticazione a sfida). Tutto il resto — fattori, protocolli, 2FA — ruota attorno a questi tre punti d'ancoraggio.

La visione d'insieme: la regola AAA

Il punto di partenza del modulo è la distinzione tra tre concetti che sembrano una cosa sola ma non lo sono.

Autenticazione è l'attribuzione certa dell'identità di un soggetto che utilizza le risorse. Normalmente include una *identificazione* preliminare (dichiari di essere Alice), ma la distinzione è critica: l'identificazione è il nome che dichiari, l'autenticazione è la prova che quel nome è davvero il tuo. Errore comune — e Prandini lo sottolinea esplicitamente — usare elementi *identificativi* come “segreti”: il nome utente, l'indirizzo email, il codice fiscale sembrano oscuri ma non lo sono. Un attaccante che ottiene il tuo username non ha ancora superato l'autenticazione, ma un sistema che usa solo il nome utente come prova di identità è già compromesso.

Autorizzazione è la verifica dei diritti di un soggetto di compiere una determinata azione su un determinato oggetto. È una decisione esplicita di concessione o negazione del permesso. Si viene dopo l'autenticazione: prima si stabilisce *chi sei*, poi si decide *cosa puoi fare*. In Linux la distinzione è netta: PAM gestisce l'autenticazione, i permessi su file e `sudo` gestiscono l'autorizzazione.

Auditing è il tracciamento affidabile di tutte le decisioni di autenticazione e autorizzazione. Il compromesso è difficile: più dettagliato è il log, più è utile per le indagini forensi, ma più impatta le prestazioni e la privacy.

I quattro fattori: conosce, possiede, è, fa

L'autenticazione si basa su ciò che solo l'utente conosce, possiede, è fisicamente, o fa (posizione). Nel modello Prover/Verifier del PDF, il Prover (P) deve dimostrare al Verifier (V) di conoscere un segreto — la scelta del *metodo* con cui P dimostra questo determina tutto ciò che segue.

Qualcosa che si conosce (password, PIN, risposta segreta) è il metodo più diffuso e più attaccato. Il suo valore è nella segretezza: se la password fuoriesce dal canale o dal database del Verifier, il fattore è bruciato.

Qualcosa che si possiede (carta bancomat, hard token, Yubikey) sposta la garanzia dal software all'hardware fisico. La Yubikey, che tornerà nel contesto di FIDO U2F, è l'esempio concreto: non si può clonare via rete.

Qualcosa che si è fisicamente (biometrico: iride, impronta digitale) ha il vantaggio dell'unicità e lo svantaggio dell'irrevocabilità — se la tua impronta digitale viene compromessa, non puoi cambiarla come una password.

Qualcosa che si fa o dove si è (GPS, geolocalizzazione, centralizzata auto) è il fattore contestuale: non si autentica con la posizione in sé, ma si usa come segnale di anomalia (login da un paese diverso dal solito).

/etc/shadow: dove vive il segreto sul lato server

Il file concreto più importante di questo modulo è `/etc/shadow`. Quando su una VM Debian (SysAdmin) o Kali (Security) guardi l'entry di un utente in `/etc/shadow`, vedi qualcosa del tipo:

```
$6$ViDM2ItuaSNPBxf0$I40Uoaqu0.OiFafaIVeMazZpIisBV.CC76...
```

Il campo si legge in tre parti separate da `$`: `$6$` è l'identificatore dell'algoritmo hash usato (6 = SHA-512), `ViDM2ItuaSNPBxf0` è il **salt**, il resto è il **fingerprint** calcolato sulla concatenazione `password || salt`.

Il meccanismo di hashing risponde a tre requisiti: il Verifier non deve conoscere la password in chiaro, il furto del file di password deve essere inefficace, ma il Verifier deve riuscire a discriminare una password corretta da una errata. La soluzione è memorizzare non la password ma la sua impronta — la funzione hash è a senso unico: facile da calcolare, difficile da invertire. Al login, il sistema calcola `hash(password_inserita || salt)` e confronta con il fingerprint memorizzato.

Il **salt** è una variazione random generata quando scegli la password. Serve a rompere la simmetria: due utenti con la stessa password hanno fingerprint diverse, e non si possono usare tabelle precalcolate (rainbow tables) perché bisognerebbe precalcolarle per ogni possibile salt. Il salt cambia ad ogni rinnovo della password: stessa password → fingerprint completamente diversa.

Però il salt non basta contro gli **attacchi offline**: chi ruba `/etc/shadow` può tentare dizionari lentamente sul proprio hardware, senza limite di tentativi. La contromisura essenziale è che la password non sia facile da indovinare (alta entropia).

Il **pepper** è una variante del salt che vive in un HSM (Hardware Security Module) anziché nel database: anche se l'attaccante ruba il file `/etc/shadow`, non ha il pepper, e il fingerprint che vede è calcolato su `password || pepper`, che non può replicare. Difesa in profondità.

Threat model (attaccante): chi ottiene `/etc/shadow` si trova in una posizione di vantaggio — può fare brute force offline a piacere. Strumenti come `hashcat` o `john` sfruttano esattamente questo. La velocità dell'attacco dipende dall'algoritmo: SHA-512 è più lento di MD5, ed è una scelta deliberata.

Threat model (difensore): il difensore controlla: la scelta dell'algoritmo (SHA-512 o bcrypt, mai MD5); la lunghezza minima della password; l'aggiunta di salt (già incluso in `/etc/shadow` moderno); i permessi sul file (leggibile solo da root — se `/etc/shadow` è world-readable, la partita è già finita).

Scelta delle password: l'entropia conta, non la complessità percepita

Il fumetto xkcd nel PDF (p. 11) è esplicito: `Tr0ub4dor&3` sembra complessa ma ha circa 28 bit di entropia — difficile da ricordare per un umano, relativamente facile per una macchina. `correct horse battery staple` ha 44 bit di entropia — facile da ricordare, difficile per una macchina.

Il punto teorico è che le stime valgono solo se le scelte sono *perfettamente casuali*. Valutare “a occhio” la complessità di una password è fuorviante. I sistemi LLM in particolare tendono a generare password che seguono distribuzioni statisticamente prevedibili — non sono buoni generatori di entropia.

Le **cattive pratiche** mostrate nel PDF (lunghezza massima troppo bassa, caratteri speciali non ammessi, rifiuto di password “troppo lunghe”) riducono lo spazio delle password possibili, paradossalmente indebolendo la sicurezza pur rispettando i requisiti formali.

La soluzione pratica è il **password manager**: un database cifrato con una sola passphrase forte, che genera e memorizza password casuali uniche per ogni sito. Cruciale perché una password ottenuta da un leak su un sito è immediatamente usata su tutti gli altri dove è stata riutilizzata (*credential stuffing*).

Autenticazione passiva vs attiva: il problema del canale

Nell'**autenticazione passiva**, P e V concordano il segreto e lo memorizzano; quando P si autentica, invia il segreto a V che lo confronta con la propria copia. È il modello della password classica.

I problemi sono strutturali: se il segreto viaggia in chiaro, un attaccante passivo sul canale lo intercetta. Se viaggia offuscato ma è sempre lo stesso, basta catturarlo e riprodurlo (*replay attack*). Servono protocolli più sofisticati, oppure un canale cifrato bene (TLS). Il secondo problema è che V possiede una copia del segreto: se V viene compromesso (furto di `/etc/shadow`), tutti i segreti sono a rischio.

Nell'**autenticazione attiva**, P convince V di possedere il segreto senza svelarlo, inviando ogni volta un dato *diverso*. Due proprietà fondamentali: il furto del dato di confronto da V diventa inutile (non è il segreto, è un derivato), e il furto dal canale è inutile per autenticazioni future (il dato è già consumato). L'attenzione va mantenuta però per l'*uomo nel mezzo* (MITM): se l'attaccante è attivo sul canale può impersonare V verso P e usare le risposte di P per autenticarsi verso V.

S-KEY One-Time Password: hash come ratchet

S-KEY è l'esempio più elegante di autenticazione attiva basata su password. Il Prover conosce un segreto N ; il Verifier viene inizializzato con $h^k(N)$, cioè il risultato di applicare k volte la funzione hash a N . La prima autenticazione: P invia $h^{(k-1)}(N)$. V verifica banalmente che $h(h^{(k-1)}(N))$

= $h^k(N)$ — calcolare un hash è facile. Poi V scarta $h^k(N)$ e ricorda $h^{(k-1)}(N)$ come nuovo riferimento.

Il meccanismo è un *ratchet* a senso unico: chi osserva $h^{(k-1)}(N)$ sul canale non può risalire a $h^{(k-2)}(N)$ perché l'hash è difficile da invertire. Ogni autenticazione consuma un “gettone” della catena. Dopo k autenticazioni il sistema va reinizializzato (esistono varianti senza limite).

Sfida e risposta: SSH e la chiave privata come segreto

I sistemi a sfida e risposta sono tipicamente implementati con crittografia asimmetrica — SSH è il caso concreto che hai già usato in SysAdmin. Il Prover possiede una chiave privata (PRIV_P); il Verifier conosce la chiave pubblica corrispondente (PUB_P, memorizzata in `~/.ssh/authorized_keys`).

Il protocollo: V genera un numero casuale (nonce), lo cifra con PUB_P e lo invia come sfida. P decifra con PRIV_P — può farlo solo chi possiede la chiave privata — e rimanda il nonce a V. V confronta: se i valori coincidono, P ha dimostrato di possedere PRIV_P senza averla mai inviata sul canale.

Questo è il motivo per cui l'autenticazione SSH a chiave pubblica è più robusta della password: anche intercettando tutto il traffico, l'attaccante ottiene solo il nonce cifrato e la risposta — nulla di riutilizzabile per autenticazioni future. L'unico rischio è compromissione della chiave privata sul dispositivo di P.

/etc/pam.d/: il framework che collega tutto a Linux

PAM (Pluggable Authentication Modules) è il livello che su Linux mette insieme tutto ciò che abbiamo visto. Qualsiasi programma che deve autenticare un utente — `login`, `ssh`, `sudo`, `su` — non implementa la logica di autenticazione da solo: la delega a PAM, che la risolve caricando moduli configurabili per programma.

La configurazione vive in `/etc/pam.d/`, un file per ogni programma (es. `/etc/pam.d/sshd`, `/etc/pam.d/sudo`). Ogni riga ha la forma:

```
module-type    control-flag    module-path    [arguments]
```

module-type determina quale aspetto del ciclo di autenticazione il modulo gestisce: `auth` (verifica dell'identità, es. confronto password con `/etc/shadow`), `account` (controlli non legati alla password, es. restrizioni orarie o di gruppo), `session` (setup e cleanup della sessione utente), `password` (aggiornamento delle credenziali).

control-flag è il cuore della logica di stack: più moduli si combinano e il flag determina come il successo o fallimento di ciascuno influenza il risultato finale. Il flag **required** significa che il modulo *deve* passare perché lo stack riesca, ma l'esecuzione continua verso i moduli successivi anche in caso di fallimento (non si rivela subito il motivo del rigetto). Il flag **requisite** è simile ma interrompe lo stack immediatamente in caso di fallimento. Il flag **sufficient** ferma lo stack con successo se il modulo passa (e nessun **required** precedente ha già fallito). Il flag **optional** non influenza il risultato finale a meno che sia l'unico modulo presente.

L'ordine dei moduli nello stack non è decorativo: con `required pam_unix.so` seguito da `sufficient pam_krb5.so`, lo stack riesce solo se `pam_unix` riesce (il `required` ha la precedenza anche sul `sufficient`). Invertendo l'ordine — `sufficient pam_krb5.so` poi `required pam_unix.so` — lo stack riesce se *uno dei due* riesce. Stessa configurazione, logica completamente diversa.

I **moduli comuni** che un difensore incontra: - `pam_unix.so`: autenticazione Unix classica via `/etc/passwd` e `/etc/shadow` - `pam_tally.so`: conta i tentativi falliti e può bloccare l'account dopo N tentativi — la difesa principale contro il brute force online - `pam_cracklib.so`: verifica la robustezza della nuova password durante il cambio (usato nel tipo `password`) - `pam_nologin.so`: se `/etc/nologin` esiste, solo root può accedere — usato per mettere un sistema in manutenzione - `pam_deny.so`: sempre fallisce; il file `/etc/pam.d/other` con `pam_warn + pam_deny` per tutti i tipi crea una configurazione *default-deny* che blocca l'autenticazione di qualsiasi programma senza configurazione PAM dedicata

Threat model: un attaccante che può modificare `/etc/pam.d/` ha già compromesso il sistema (richiede root). Ma misconfigurare PAM è un vettore di privilege escalation indiretto: in S11 vedrai come misconfigurazioni di `sudo` e dei permessi Linux sfruttano esattamente la catena `auth` → `account` → `session` che PAM gestisce.

2FA vs 2SA vs MFA: la distinzione che l'esame testa

Questa è la distinzione più sottile del modulo e uno dei candidati forti per il quiz teorico.

2FA (Two-Factor Authentication) richiede due fattori di autenticazione che siano *distinti* — provenienti da due categorie diverse (conosce + possiede, oppure possiede + è). L'elemento chiave è la distinzione categoriale.

2SA (Two-Step Authentication/Verification) richiede due passi ma non necessariamente due fattori distinti. Il codice OTP ricevuto via SMS — il classico “ti abbiamo inviato un codice” — aggiunge un secondo passo ma non è considerato autenticazione di secondo fattore perché SMS e email sono vulnerabili ad attacchi MITM: chi controlla il canale può intercettare il codice. Non è quindi “qualcosa che si possiede” in senso forte.

Un ulteriore caso limite: sbloccare un'app con un PIN per ottenere un token OTP è “conoscenza aggiuntiva”, non un secondo fattore distinto — sia la password che il PIN rientrano nella categoria “conosce”. **MFA (Multi-Factor Authentication)** generalizza 2FA a più di due fattori distinti.

Perché importa: il livello di protezione è profondamente diverso. La 2SA via SMS protegge contro chi ha solo la password rubata, ma non contro chi controlla la rete o il telefono. La 2FA vera con dispositivo hardware dedicato (Yubikey, hard token) protegge anche contro un attaccante che controlla il canale di comunicazione del secondo fattore — perché il secondo fattore non transita su quel canale.

OTP e TOTP: l'OTP (One Time Password) è valida per un solo utilizzo; TOTP (Time-based OTP) aggiunge il vincolo temporale (tipicamente 5 secondi–pochi minuti). Il token TOTP su app (Google Authenticator, etc.) è un esempio di “qualcosa che si possiede” — il segreto condiviso che genera il codice è memorizzato nell'app del dispositivo.

FIDO: lo standard che porta la crittografia a chiave pubblica al grande pubblico

FIDO Alliance (Fast IDentity Online) è un consorzio (Google, Microsoft, Yubico e altri) che ha sviluppato standard per rendere l'autenticazione forte usabile su larga scala.

FIDO UAF (Universal Authentication Framework) è lo standard per l'autenticazione *senza password*. L'utente si autentica localmente sul proprio dispositivo con biometria o PIN; il dispositivo usa quella verifica locale per sbloccare una chiave privata, che firma la sfida emessa dal server FIDO UAF. Le informazioni di sicurezza (chiave privata, biometria) non lasciano mai il dispositivo — il server vede solo la firma e la chiave pubblica.

FIDO U2F (Universal Second Factor, evoluto in FIDO2/WebAuthn) rafforza e semplifica la 2FA usando dispositivi hardware USB, NFC o Bluetooth. La **YubiKey** ne è l'implementazione più nota: un dispositivo fisico che implementa FIDO2/WebAuthn, U2F, smart card, OpenPGP, OTP su USB-A, USB-C o NFC. Il flusso U2F è: il browser invia la sfida al device → l'utente tocca il pulsante fisico (presenza umana verificata) → il device firma con la propria chiave privata → il server verifica.

La protezione anti-phishing integrata in U2F è un vantaggio chiave rispetto a OTP via SMS: la YubiKey esegue un *origin-check* — verifica il dominio del sito prima di firmare la sfida. Un sito di phishing (es. `g00gle.com`) riceve una firma valida solo per quel dominio, non usabile su `google.com`.

Anche la YubiKey non è immune: nel 2018 Vervier e Orrù hanno dimostrato come la feature WebUSB di Chrome (che permette a pagine web di comunicare direttamente con device USB via JavaScript) potesse essere usata per aggirare l'origin-check della YubiKey NEO, inviando richieste U2F direttamente all'interfaccia CCID del device. Il punto non è che la YubiKey è insicura, ma che la sicurezza dell'hardware dipende dall'isolamento del contesto software che lo circonda.

Connessioni

Con SysAdmin 2A: `pam_unix.so` è il modulo che legge `/etc/shadow` ogni volta che esegui `su`, `sudo` o fai login sulla VM Debian. I permessi su `/etc/shadow` (leggibile solo da root, `-rw-r-----` root shadow) sono la prima linea di difesa contro il furto offline.

Con SysAdmin 4C (LDAP): l'integrazione LDAP per l'autenticazione centralizzata funziona esattamente via PAM — si aggiunge `pam_ldap.so` allo stack di `/etc/pam.d/` per ogni servizio che deve autenticare via directory. L'esempio nel PDF PAM mostra proprio `auth sufficient /lib/security/pam_ldap.so`.

Con S1 (enumerazione): Nmap ha trovato le porte — porta 22 (SSH), porta 21 (FTP), porta 80 (HTTP). Ora S2 spiega cosa succede dopo: il banner grabbing di S1 rivela la versione del servizio, ma l'accesso reale richiede superare il meccanismo di autenticazione. SSH con password è vulnerabile al brute force (contromisura: `pam_tally`, `fail2ban`); SSH con chiave pubblica lo è molto meno.

Con S3/S4: SQL injection (S3) può bypassare l'autenticazione applicativa controllando la query di verifica della password. Buffer overflow (S4) può sovrascrivere variabili di stato in memoria, inclusi flag di autenticazione. In entrambi i casi, l'autenticazione è il perimetro che si cerca di aggirare.

Con S9 (Demoni e Autorizzazione): PAM gestisce l'autenticazione; DAC/MAC/RBAC (S9) gestiscono l'autorizzazione. I due livelli AAA si vedono separati e configurabili.

Domande di autoverifica (stile quiz teorico)

Ricorda: nell'esame c'è penalità per risposta sbagliata. Se non sei sicuro al 70%, non rispondere.

1. Un sistema chiede username e poi password. Il momento in cui inserisci lo username è: - a) Autenticazione - b) Autorizzazione - c) Identificazione - d) Auditing
2. Vero o Falso: il salt memorizzato in `/etc/shadow` protegge efficacemente contro attacchi offline (brute force dopo furto del file). > **Falso** — il salt impedisce attacchi con tabelle precalcolate (rainbow tables) e rivela quando due utenti hanno la stessa password, ma non ferma chi può fare brute force direttamente sul file rubato. La protezione contro attacchi offline è la robustezza della password stessa (alta entropia).
3. Un utente riceve il codice di verifica via SMS dopo aver inserito la password. Questa è: - a) 2FA vera, perché usa due passi - b) 2SA, non 2FA, perché SMS è vulnerabile a MITM - c) MFA, perché combina due fattori - d) Autenticazione passiva
4. Nel control-flag PAM, qual è la differenza tra **required** e **requisite**? > Con **required**, in caso di fallimento lo stack continua verso i moduli successivi (ma il risultato finale sarà fallimento — non si rivela subito il modulo che ha fallito). Con **requisite**, il fallimento termina lo stack immediatamente con fallimento.
5. Vero o Falso: in S-KEY OTP, un attaccante che intercetta $h^{(k-1)}(N)$ sul canale può usarlo per la prossima autenticazione. > **Falso** — dopo che P ha inviato $h^{(k-1)}(N)$, V lo registra come nuovo riferimento e lo scarta come credenziale valida. L'autenticazione successiva richiede $h^{(k-2)}(N)$, che l'attaccante non può calcolare perché l'hash è difficile da invertire.
6. Vero o Falso: la YubiKey con FIDO U2F protegge intrinsecamente contro qualsiasi attacco, incluso quello via WebUSB. > **Falso** — come dimostrato nel 2018, WebUSB può essere usato per aggirare l'origin-check della YubiKey inviando richieste all'interfaccia CCID. La sicurezza del token hardware dipende dall'isolamento del contesto software che lo circonda.

Riepilogo

AAA non è una sola cosa: identificazione (chi dici di essere) autenticazione (prova che lo sei) autorizzazione (cosa puoi fare). Confonderle è il punto di partenza di molte vulnerabilità.

/etc/shadow + salt: il difensore memorizza `hash(password || salt)`, non la password; il sale impedisce le rainbow table ma non il brute force offline — la vera protezione è l'entropia della password.

2FA 2SA: SMS come secondo passo è 2SA, non 2FA vera — i fattori devono essere categorialmente distinti (conosce + possiede, non conosce + conosce). PAM su Linux è il framework che assembla questi meccanismi; l'ordine e i control-flag degli stack determinano la logica risultante.

Prossimo passo: questa lezione si legge prima di accendere la VM. Non c'è lab specifico per S2 — il contenuto alimenta il quiz teorico. Continua con `/lezione S3` (Web Security) o `/lab 3D` per proseguire con SysAdmin.

Collegati

Hub: [\[\[master_map_studio\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[metodo_studio_esami_pratici\]\]](#)